CO3 - ASSESSMENT TOOL

Exception Handling in Thread Pool Worker Threads: execute() vs submit()/Future

Tool Name	CO3 - Assessment
Name	M. Yogeshwaran
Reg No	192524363
Course Name	Java Programming
Course Code	CSA0924

1. What Happens When an Exception Is Thrown Inside run()

Every thread in a thread pool (an ExecutorService backed by a ThreadPoolExecutor) is a worker thread that repeatedly pulls Runnable/Callable tasks off a shared work queue and invokes their run() (or call()) method. What happens when that code throws an exception depends on the type of exception, and on how the task was submitted to the pool.

1.1 Unchecked Exceptions (RuntimeException / Error) in a Runnable

A Runnable's run() method cannot declare checked exceptions, so only unchecked exceptions (RuntimeException) or Errors can propagate out of it. When such an exception escapes run() uninterrupted:

- The worker thread's run loop, inside ThreadPoolExecutor.runWorker(), catches the throwable, calls afterExecute(task, throwable), and then lets the thread terminate.
- The pool detects the dying worker thread and replaces it with a brand-new worker thread, so the pool's configured size is maintained.
- By default the exception is printed to System.err via the thread's uncaught exception handler (Thread.UncaughtExceptionHandler) — it is NOT silently swallowed, but it is also not delivered back to the code that submitted the task, because that code already returned from execute() long before the task ran.
- Other queued/running tasks in the pool are unaffected; only the one worker thread that hit the exception is torn down and replaced.

1.2 Key Point — the Pool Survives, the Thread Does Not

A ThreadPoolExecutor never lets a single misbehaving task crash the whole pool. It isolates each task's execution inside the worker's run loop with a try/catch, so one thrown exception only costs the pool one worker thread, which is immediately replenished.

2. execute() vs submit()

Both methods hand a task to the pool for asynchronous execution, but they differ in return type, exception signature, and — most importantly — how exceptions become visible to the caller.

Aspect	execute(Runnable)	submit(Runnable/Callable)
Declared in	Executor interface	ExecutorService interface
Return value	void — nothing returned	Future<?> or Future<V> — a handle to the pending result
Accepts	Runnable only	Runnable, Runnable+result, or Callable
Exception propagation	Not returned to caller. Handled by the worker's afterExecute()/uncaught-exception handler; caller has no direct way to know a task failed.	Captured, stored inside the Future, and only thrown to the caller when Future.get() is invoked (wrapped in ExecutionException).
Fire-and-forget?	Yes — natural fit when you don't need a result or failure notification	No — designed for cases where you want to check completion, cancel, or retrieve a result/error

3. The Role of Future

When a task is given to submit(), the executor wraps it in a FutureTask before queuing it. FutureTask's run() method itself contains a try/catch: it invokes the underlying Callable/Runnable, and if it throws, FutureTask catches the throwable and stores it as the task's outcome instead of letting it escape to the worker thread.

- Because FutureTask swallows the exception internally, the worker thread's run loop sees a normal return — the worker thread is NOT killed and does not need to be replaced.
- The stored exception stays dormant until the caller calls future.get(), at which point it is rethrown wrapped in an ExecutionException, with the original exception available via getCause().
- future.get(timeout, unit) additionally can throw TimeoutException if the task hasn't finished; get() also throws InterruptedException if the waiting thread is interrupted, and CancellationException if the task was cancelled.
- If get() is never called, an exception thrown inside the task is effectively lost — no stack trace is printed anywhere, which is a common source of silently failing submit()-based code.

3.1 Example

```
ExecutorService pool = Executors.newFixedThreadPool(4);

// execute(): fire-and-forget, exception goes to the
// default uncaught exception handler, not to this code
pool.execute(() -> { throw new RuntimeException("boom-execute"); });

// submit(): exception is captured inside the Future
Future<?> future = pool.submit(() -> { throw new RuntimeException("boom-
submit"); });
try {
    future.get();               // rethrown here
} catch (ExecutionException e) {
    System.out.println("Caught: " + e.getCause());
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
```

4. Summary

- execute(): exception surfaces on the worker thread itself, kills and replaces that thread, and is only visible via the thread's uncaught-exception handler (e.g. console stack trace) — the calling code never sees it directly.
- submit(): exception is caught by FutureTask, stored inside the Future, and the worker thread survives; the caller must call Future.get() to observe the failure (as an ExecutionException).
- Pool integrity: regardless of which method is used, the ThreadPoolExecutor as a whole keeps running; at most one worker thread is affected, and it is transparently replaced when using execute().
- Best practice: use submit() with Future (or CompletableFuture) when task failures must be detected or logged; always call get() or otherwise inspect the Future, and consider a custom Thread.UncaughtExceptionHandler or a wrapped Runnable when using execute() to avoid silent failures.